

Experiment-1-Beginner-1

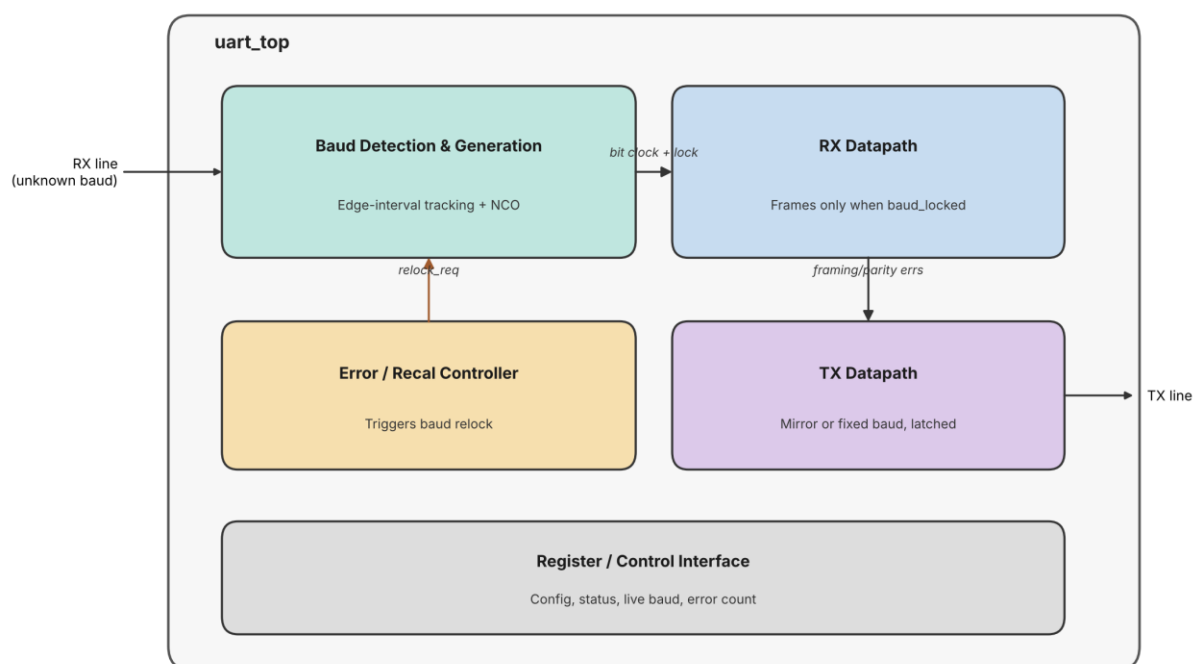
Title: FPGA-Based Self-Correcting UART with Real-Time Baud Detection

Objective:

The objective of this experiment is to design and implement a hardware-oriented, self-correcting UART on an FPGA that automatically detects an unknown incoming baud rate in real time, without the baud rate being configured in advance. The receiver measures the bit period of incoming serial data using edge-interval tracking, generates an oversampled sampling clock through a phase-accumulator NCO, and continuously re-validates its baud estimate as data is received, re-acquiring lock automatically if framing or parity errors indicate the estimate has drifted.

The complete UART core is described using Verilog HDL, validated through ModelSim RTL simulation, and implemented on the Terasic DE10-Nano FPGA board (Cyclone V SE, 5CSEBA6U23I7). The design explores auto-baud acquisition, oversampled receiver synchronization, and self-correcting recalibration to make reliable UART communication possible even when the host baud rate is unknown.

Block Diagram:



The system accepts an asynchronous serial input (RX line) at an unknown baud rate. The line first passes through a synchronizer, after which the Baud Detection & Generation block measures the minimum interval between successive edges to estimate the incoming bit period and uses it to drive an oversampled sampling clock. The RX Datapath samples each bit at its midpoint using this clock, but only frames and delivers a byte once the baud rate has genuinely locked. Framing and parity errors are reported to the Error / Recalibration Controller, which forces the baud detector to re-acquire lock if errors persist. A Register / Control Interface exposes configuration and live status -- detected baud, lock state, and error counts. The TX Datapath transmits using either the same auto-detected baud rate or a fixed configured rate.

RTL Design:

The UART core is implemented entirely in Verilog HDL as a set of cooperating modules, integrated under a single top-level module (uart_top.v). Each block was independently verified before being combined into the complete design.

1. rx_line_sync.v

Provides a double-flop synchronizer for the raw, asynchronous RX pad signal, together with edge-detection pulses used by the rest of the receiver. This is the only block that touches the raw pad signal directly, isolating the rest of the design from metastability risk.

2. baud_rate_detector.v

Implements auto-baud acquisition using a minimum edge-interval tracking method: it continuously measures the shortest interval between edges over a rolling window, filters the result against a configurable tolerance, and declares a lock once a run of consecutive measurements agree. This continuous re-arming is what allows the design to correct for baud drift in real time rather than only at power-on.

3. baud_nco.v

Converts the detected bit period into an oversampling clock using a phase-accumulator (numerically controlled oscillator) architecture, resynchronizing its phase to every genuine start-bit edge so that oversample timing stays consistent from frame to frame.

4. uart_rx.v

Implements the receiver state machine: it samples the synchronized RX line at the midpoint of each bit using the oversampled clock, shifts received bits into a register, and checks framing and parity. The receiver only frames and delivers a byte once the baud rate detector has genuinely locked, so that data received during acquisition is used only to help lock on and is never delivered as if it were valid.

5. uart_tx.v

Implements the transmitter state machine, shifting out start, data, optional parity, and stop bits at the configured or auto-detected bit rate. The selected bit period is latched only between frames, so a mid-transmission update can never distort a byte already being sent.

6. error_controller.v

Aggregates framing, parity, and FIFO overrun errors from the receiver. Once accumulated errors cross a configurable threshold, it issues a relock request back to the baud rate detector, forcing re-acquisition rather than allowing the receiver to continue operating on a baud estimate that may no longer be valid.

7. sync_fifo.v, reg_if.v and uart_top.v

sync_fifo.v is a first-word-fall-through FIFO used for both the receive and transmit data paths. reg_if.v exposes a memory-mapped register interface for configuration and live status. uart_top.v is the top-level integration module that instantiates and wires together every block above into the complete UART core.

8. Testbench and Verification

The integrated design was verified in ModelSim using a top-level testbench, tb_de10nano_uart_top.v, running at a 20 ns (50 MHz) system clock with BIT_CLKS = 434 (approximately 115200 baud). The testbench exercised seven groups of checks: reset behavior, auto-baud acquisition, single-byte echo, multiple-byte echo, and status LED behavior, using self-checking comparisons against expected transmitted and received data.

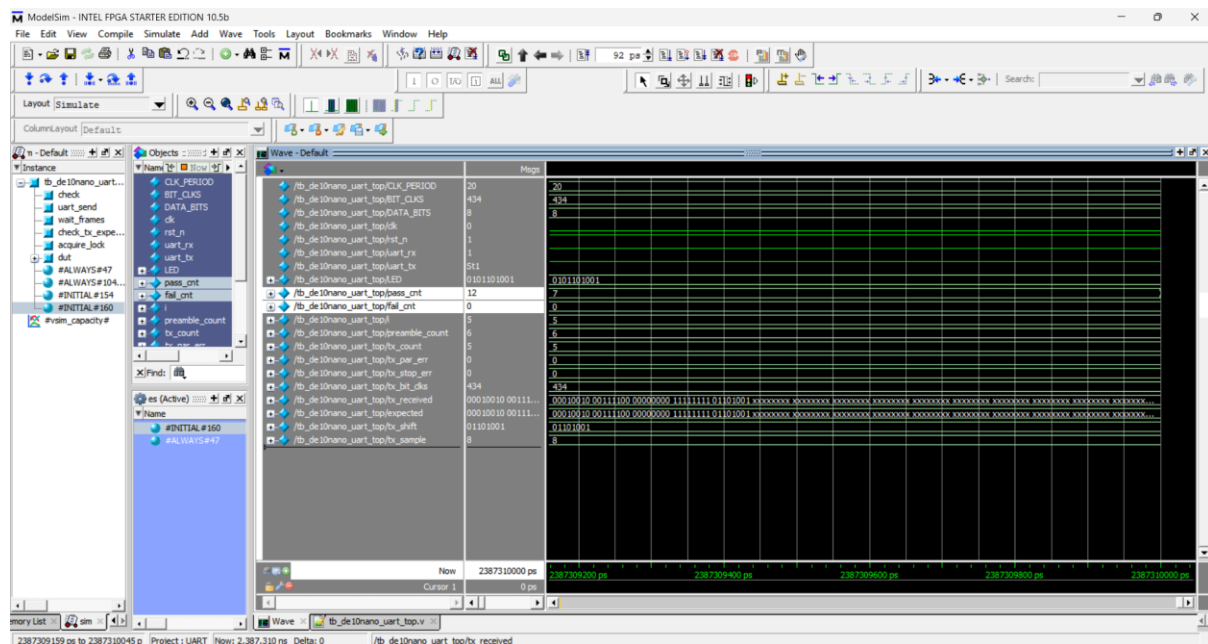
Simulation Results:

```
Transcript
File Edit View Bookmarks Window Help
Transcript
sim:/tb_de10nano_uart_top/1 \
sim:/tb_de10nano_uart_top/preamble_count \
sim:/tb_de10nano_uart_top/tx_count \
sim:/tb_de10nano_uart_top/tx_par_err \
sim:/tb_de10nano_uart_top/tx_stop_err \
sim:/tb_de10nano_uart_top/tx_bit_clks \
sim:/tb_de10nano_uart_top/tx_received \
sim:/tb_de10nano_uart_top/expected \
sim:/tb_de10nano_uart_top/tx_shift \
sim:/tb_de10nano_uart_top/tx_sample
V$IM 3> run -all

# =====
#   tb_de10nano_uart_top: start (BIT_CLKS=434, CLK=20 ns)
# =====

# --- T1: reset state ---
# [2000000] PASS: uart_tx idles high during reset
# [2000000] PASS: LED outputs are clear during reset
#
# --- T2/T3: auto-baud acquisition ---
# [781510000] PASS: LED[8] asserts after auto-baud lock
# [781510000] PASS: Auto-baud acquired within 60 preamble bytes
#
# --- T4/T5: single-byte echo ---
# [1302310000] PASS: LED[7:0] shows last received byte 0xA5
# [1302310000] PASS: Single received byte echoed correctly
# [1302310000] PASS: Echoed UART frame has a high stop bit
#
# --- T6: multiple-byte echo ---
# [2387310000] PASS: LED shows final byte of multi-byte sequence
# [2387310000] PASS: Multiple bytes echoed correctly and in order
# [2387310000] PASS: All echoed frames have high stop bits
#
# --- T7: status LEDs ---
# [2387310000] PASS: Baud-lock LED remains asserted
# [2387310000] PASS: Error LED low after valid frames
#
# =====
# SUMMARY: 12 passed, 0 failed
# *** ALL TESTS PASSED ***
# =====
# * Note: $finish      : C:/Users/pgkri/Downloads/tb_de10nano_uart_top.v(243)
# Time: 2387310 ns Iteration: 0 Instance: /tb_de10nano_uart_top
# 1
```

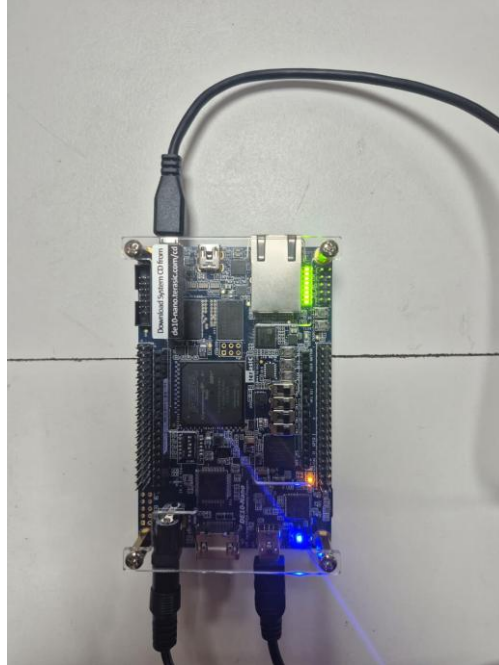
ModelSim transcript: all 12 checks passed across reset, auto-baud acquisition, single- and multi-byte echo, and status LED tests.



ModelSim waveform at end of simulation: tx_received matches expected, LED = 0101101001, pass_cnt = 12, fail_cnt = 0.

The simulation confirmed auto-baud lock (LED[8] asserting) within 60 preamble bytes, correct echo of both single and multiple received bytes with valid stop bits, and a status LED pattern (LED[7:0] = last received byte, LED[8] = baud-lock, LED[9] = error flag) that remained consistent throughout. The final transcript summary reported 12 passed, 0 failed -- all tests passed.

Hardware Implementation:



The design powered up on the Terasic DE10-Nano (Cyclone V SE, 5CSEBA6U23I7).

The verified design was deployed on the Terasic DE10-Nano board. The DE10-Nano provides a 50 MHz system clock (FPGA_CLK1_50 / FPGA_CLK2_50) and two 40-pin GPIO expansion headers (GPIO_0 and GPIO_1) with 36 user-accessible FPGA pins each. The board's onboard USB-UART bridge (FT232R) is wired to the Hard Processor System rather than to the FPGA fabric, so -- as with most pure-fabric UART designs on this board -- the serial RX/TX lines for this design were brought out on the GPIO header to an external 3.3V-logic USB-to-serial adapter for terminal testing, rather than through the onboard HPS USB port. The photograph above shows the board powered and running the design, with the FPGA configured and its status LEDs active.

Applications:

A self-correcting, auto-baud UART is directly applicable wherever the communication baud rate cannot be assumed in advance: embedded debug and bootloader consoles, modular test and measurement equipment that must interoperate with a variety of legacy or third-party devices, IoT sensor nodes with field-configurable host links, and general-purpose FPGA-based communication bridges. The same edge-interval detection and oversampled-NCO architecture generalizes to any asynchronous serial protocol requiring runtime clock recovery, beyond UART specifically.

Conclusion:

The experiment successfully implemented a self-correcting UART core with real-time auto-baud detection, verified in ModelSim with a self-checking testbench covering reset, acquisition, echo, and status behavior, and deployed on the Terasic DE10-Nano FPGA board. All 12 simulation checks passed, confirming correct auto-baud lock, accurate byte echo, and consistent status reporting. The work provided practical experience in asynchronous clock recovery, phase-accumulator NCO design, and FPGA hardware bring-up.